ESTIMATIVA E MÉTRICAS SOFTWARE

AULA 6

Prof.ª Rosemari Pavan Rattmann

CONVERSA INICIAL

Nesta abordagem, retomaremos os conceitos de métricas e estimativas de software, destacando sua importância na redução de incertezas, na definição de prioridades e no acompanhamento do progresso dos projetos.

Em seguida, analisaremos as ferramentas para coleta e análise de métricas, essenciais para transformar dados brutos em informações significativas. Vamos apresentar diversas tecnologias, considerando seus usos práticos, com foco na forma como elas auxiliam equipes no monitoramento da qualidade do código, no desempenho do sistema e na identificação de melhorias necessárias.

Dando continuidade, discutiremos as melhores práticas no uso de métricas, conectando teorias a aplicações práticas. Vamos destacar estratégias para assegurar a confiabilidade dos dados, como padronização e revisão contínua de métricas, além da relevância da análise qualitativa no contexto da avaliação de projetos complexos.

Posteriormente, vamos explorar o uso de ferramentas para estimativas de software, apresentando técnicas inovadoras que facilitam o planejamento em ambientes ágeis. Essa abordagem enfatiza a colaboração das equipes e o ajuste dinâmico de estimativas para atender às necessidades do projeto e do cliente.

Por fim, examinaremos o papel crítico das estimativas no desenvolvimento, compra e manutenção de software, destacando sua relevância na previsão de custos, esforços e cronogramas. Ao integrar métricas e estimativas, discutiremos como as empresas podem alinhar estratégias de tecnologia com objetivos organizacionais.

Cada um desses tópicos será apresentado em uma sequência que busca não apenas transmitir conhecimento, mas também fomentar a reflexão sobre a aplicação prática desses conceitos no contexto do desenvolvimento de software.

Vamos lá?

TEMA 1 – MÉTRICAS E ESTIMATIVAS DE SOFTWARE (SÍNTESE DO CONHECIMENTO)

Métricas e estimativas de software são instrumentos indispensáveis no contexto de projetos que envolvem desenvolvimento, manutenção ou aquisição de sistemas. As métricas, por sua natureza quantitativa, fornecem dados sobre

aspectos como qualidade, desempenho e produtividade, enquanto as estimativas ajudam a prever esforços, custos e prazos necessários para alcançar os objetivos estabelecidos. Quando utilizadas de forma integrada, essas ferramentas tornam-se aliadas fundamentais para um planejamento eficiente, com boa gestão de recursos e antecipação de possíveis desafios, contribuindo diretamente para o sucesso dos projetos.

Entre os benefícios mais evidentes dessas práticas, está a redução de incertezas, o alinhamento de expectativas entre os diversos stakeholders e a capacidade de identificar problemas potenciais ainda nas fases iniciais do projeto. Métricas confiáveis permitem monitorar o progresso e avaliar a saúde do trabalho em andamento, enquanto estimativas bem estruturadas possibilitam a priorização adequada de tarefas e a utilização otimizada de recursos. Apesar dessas vantagens, é importante reconhecer os riscos associados, especialmente quando as estimativas são baseadas em dados insuficientes ou subjetivos, o que pode levar a projeções imprecisas, comprometer a tomada de decisões e gerar atrasos e custos adicionais.

A implementação de boas práticas, como a padronização de métodos, o uso de ferramentas automatizadas e a documentação criteriosa, é fundamental para minimizar esses riscos. Ao fundamentar as métricas e estimativas em dados históricos reais e práticas consistentes, as equipes conseguem alinhar as projeções às demandas do projeto, tornando decisões mais assertivas e aumentando a probabilidade de entregar soluções de qualidade, dentro dos prazos e orçamentos previstos.

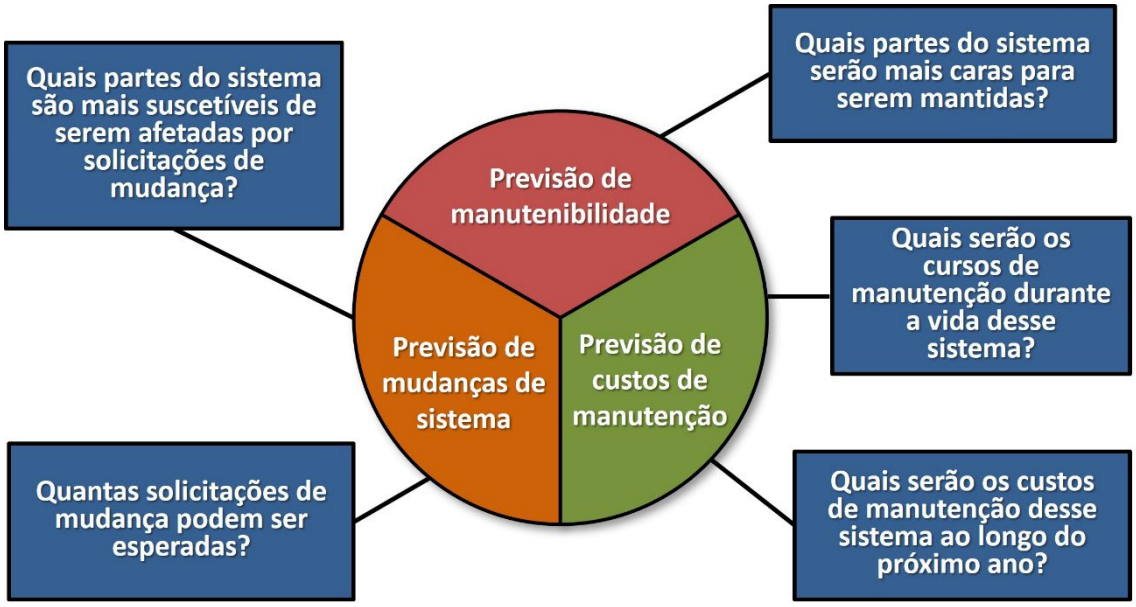
1.1 Uso de métricas de software

O uso de métricas no gerenciamento de projetos desempenha um papel essencial, ao fornecer informações concretas e objetivas que permitem aos gestores embasarem suas decisões da forma mais eficiente e confiável. As métricas funcionam como ferramentas indispensáveis para o planejamento, a estimativa de recursos e o esforço, considerando ainda o monitoramento do progresso e o controle de variáveis críticas ao longo do ciclo de vida do projeto. Elas transformam dados brutos em insights significativos, permitindo que os gerentes identifiquem riscos potenciais, acompanhem o desempenho em tempo real e ajustem estratégias para garantir o cumprimento dos objetivos dentro dos prazos e orçamentos estabelecidos. Esse processo promove maior

previsibilidade, transparência e eficiência, contribuindo para a redução de incertezas e o aumento da qualidade na entrega dos resultados esperados.

Conforme definido pelo IEEE, as métricas são uma medida quantitativa que avalia o grau em que um sistema, componente ou processo atende a determinados atributos. Elas têm ampla aplicação, abrangendo áreas como estimativas de custo, controle de qualidade, monitoramento de progresso e avaliação de complexidade, sendo cruciais para uma gestão eficiente dos projetos (IEEE). A Figura 1 mostra possíveis previsões de manutenção de um software, como exemplo de avaliação de métricas.

Figura 1 – Possíveis previsões de manutenção de software



1.1.1 Importância das métricas no gerenciamento

As métricas são instrumentos estratégicos que ajudam a estabelecer metas mensuráveis e acompanhar o progresso em relação aos objetivos definidos. Elas permitem uma visão clara sobre os elementos críticos que afetam o desenvolvimento de software, como tamanho do sistema, complexidade dos componentes, qualidade do código e esforço necessário para a sua conclusão. Além disso, ao fornecer dados tangíveis, as métricas auxiliam na tomada de decisões mais informadas e na mitigação de riscos (Pressman, 2010).

A previsão de manutenção em sistemas de software envolve identificar e avaliar as mudanças necessárias para manter o funcionamento e a adaptabilidade de um sistema ao longo do tempo. Esse processo tem como

objetivo antecipar as partes do sistema que são mais complexas ou custosas de modificar, permitindo um planejamento eficiente, com a criação de componentes que sejam mais flexíveis e adaptáveis. Dessa forma, ao considerar a previsibilidade da manutenção, é possível reduzir custos futuros, minimizando esforços em alterações posteriores (Sommerville, 2011). Esse tipo de análise preventiva também auxilia na priorização das intervenções e nos esforços de gerenciamento, garantindo que o sistema continue a atender às necessidades de seus usuários e às demandas do mercado de forma sustentável.

Métricas previsíveis desempenham um papel fundamental nesse processo, fornecendo dados objetivos e quantificáveis, que auxiliam na identificação de áreas críticas e no cálculo de custos e esforços necessários para a manutenção.

Dentre as métricas mais aplicadas, destacam-se:

- **Pontos de Função (FPA)**: método para medir o tamanho funcional de um sistema com base nos requisitos entregues ao usuário, sendo amplamente utilizado para estimar esforço e custo (Albrech; Gaffney, 1983).
- **Complexidade Ciclomática**: mede a complexidade lógica de um programa, auxiliando no planejamento de testes e na análise de manutenibilidade (McCabe, 1976).
- **Velocidade da equipe**: em métodos ágeis, mede a quantidade de trabalho concluído em um sprint, facilitando a previsão de entregas futuras (Schwaber; Beedle, 2002).
- **Densidade de defeitos**: avalia a qualidade, ao calcular o número de defeitos por unidade de tamanho do software, contribuindo para a análise de confiabilidade (Sommerville, 2011).

1.1.2 Métricas, estimativas e tomada de decisão

O emprego de métricas no contexto de desenvolvimento, aquisição ou manutenção de software é considerado um recurso indispensável para fundamentar decisões estratégicas. Métricas proporcionam informações objetivas e quantificáveis, auxiliando na distribuição otimizada de recursos, na definição de cronogramas viáveis, no controle de custos e na avaliação contínua do desempenho de projetos PMBOK. Essa abordagem é ainda mais relevante

em ambientes caracterizados por frequentes alterações nos requisitos, pois permite que os entregáveis sejam ajustados de maneira alinhada às expectativas e necessidades dos stakeholders.

As métricas ajudam a transformar informações subjetivas em dados quantitativos, fornecendo uma base objetiva para decidir entre desenvolver um software internamente, comprar uma solução existente no mercado ou investir em sua manutenção.

No processo de escolha entre adquirir um software ou desenvolvê-lo internamente, o uso de métricas como o Custo Total de Propriedade (TCO) e a escalabilidade futura desempenha um papel essencial. Enquanto sistemas adquiridos geralmente oferecem uma implementação mais rápida, podem apresentar desafios relacionados à integração com outros sistemas ou à flexibilidade para atender a necessidades específicas. As métricas, nesse contexto, ajudam a identificar esses possíveis obstáculos antecipadamente, permitindo uma decisão mais bem informada e alinhada com as metas organizacionais. Quando se considera a manutenção de software, métricas como taxa de defeitos por módulo, densidade de defeitos e custo de manutenção por linha de código ajudam a identificar se o software está se tornando financeiramente inviável. Eventualmente, sinalizam a necessidade de substituição (Pressman, 2010).

A gestão de riscos e o alinhamento das soluções tecnológicas aos objetivos estratégicos de um negócio são medidas que auxiliam gestores do projeto com indicadores de qualidade. A frequência de mudanças no código e o grau de acoplamento entre módulos são ferramentas valiosas para avaliar o impacto de novas funcionalidades em sistemas legados, fornecendo uma visão mais clara dos riscos associados a modificações, o que permite que os gestores tomem decisões informadas e reduzam potenciais interrupções no ambiente de produção. As métricas preventivas podem assegurar que a solução desenvolvida ou adquirida esteja alinhada com as metas estratégicas da organização.

1.1.3 Desafios e limitações no uso de métricas

Apesar de suas vantagens, o uso de métricas enfrenta desafios, como a subjetividade em medições e a falta de padronização. A escolha inadequada ou a má interpretação dos dados podem levar a decisões equivocadas. Por isso, é

importante selecionar métricas alinhadas aos objetivos organizacionais e combiná-las com análises qualitativas (Kitchenham; Pfleeger; Fenton, 1996).

Métricas mal compreendidas ou aplicadas sem contexto podem levar a ações que não só falham em resolver problemas reais, como também criam novos desafios, a exemplo de sobrecarga desnecessária de equipes ou priorização equivocada de funcionalidades (Pressman, 2010).

Para minimizar esses riscos, é fundamental adotar práticas que garantam a relevância e a eficácia das métricas. Algumas recomendações incluem:

- **Alinhamento estratégico:** o alinhamento estratégico garante que as métricas selecionadas estejam diretamente conectadas aos objetivos do negócio e às metas do projeto. Isso significa que elas devem refletir os resultados que a organização deseja alcançar, como aumento de eficiência, redução de custos ou melhoria na experiência do usuário. Por exemplo, se o objetivo do projeto é melhorar o desempenho de um sistema, métricas como tempo de resposta e uso de recursos do sistema são mais relevantes. Esse alinhamento evita que esforços sejam direcionados a indicadores que não agregam valor, promovendo decisões baseadas em resultados mensuráveis e relevantes (Sommerville, 2011).
- **Contextualização:** a combinação de métricas quantitativas com análises qualitativas permite uma visão mais abrangente e precisa da situação. Dados numéricos, como número de defeitos ou desempenho do sistema, oferecem uma base objetiva. No entanto, análises qualitativas, como feedback do usuário ou insights da equipe, trazem contextos que os números isolados não capturam. Por exemplo, um sistema pode ter poucos defeitos registrados, mas os usuários podem estar insatisfeitos com a interface. Esse equilíbrio garante que as decisões levem em conta tanto os dados objetivos quanto as percepções humanas, ajudando a evitar interpretações equivocadas e alinhando o projeto às expectativas reais (Kitchenham; Pfleeger; Fenton, 1996).
- **Padronização e validação:** a padronização das métricas ajuda a criar uma linguagem comum para medições, facilitando a comparação entre projetos e ciclos de desenvolvimento. Isso é particularmente útil em organizações com múltiplas equipes ou unidades, em que métricas consistentes permitem benchmarking e identificação de melhores

práticas. A revisão contínua das métricas é essencial para mantê-las alinhadas com a evolução do projeto e as necessidades do negócio. À medida que o projeto avança, novas prioridades podem surgir, e as métricas precisam ser ajustadas para refletir essas mudanças (Pressman, 2010). Por exemplo, durante as fases iniciais, pode ser mais importante medir a velocidade de entrega; já na fase final, o foco pode estar na qualidade e estabilidade do produto. Revisar periodicamente as métricas ajuda a identificar falhas ou lacunas nos indicadores, ajustando-os para garantir decisões mais acertadas e alinhadas aos objetivos organizacionais.

TEMA 2 – FERRAMENTAS PARA COLETA E ANÁLISE DE MÉTRICAS

O desenvolvimento de um software moderno é um processo complexo, que envolve uma série de atividades interligadas, desde concepção e planejamento até implementação, teste e manutenção. Para auxiliar os desenvolvedores nessa jornada, diversas ferramentas foram criadas, cada uma com um propósito específico, sempre considerando que a coleta e a análise de métricas de software são muito importantes para garantir a qualidade, o desempenho e a manutenibilidade de um produto. Essas métricas fornecem insights valiosos sobre o código, o processo de desenvolvimento e o produto final, permitindo que as equipes tomem decisões mais informadas e otimizem seus processos.

Exploraremos diversas ferramentas utilizadas no mercado de desenvolvimento de software para coletar e analisar métricas, apresentando características, funcionalidades e exemplos de aplicação. No final, apresentaremos uma tabela comparativa, para facilitar a escolha da ferramenta mais adequada para cada projeto.

As ferramentas funcionam coletando dados sobre o software, como o código fonte, os resultados dos testes, o desempenho da aplicação e a utilização dos recursos do sistema. Em seguida, esses dados são analisados e apresentados em relatórios e gráficos, permitindo que os desenvolvedores identifiquem problemas, otimizem o código e avaliem a qualidade do software.

2.1 Conceitos importantes para a avaliação das ferramentas

É importante entender alguns conceitos para que a descrição das ferramentas fique clara. A lista a seguir sugere alguns conceitos de análise:

- **CI/CD (integração contínua/entrega contínua): i**magine um restaurante. Os cozinheiros (desenvolvedores) estão sempre preparando novos pratos (código). Para que esses pratos cheguem à mesa do cliente (usuário final) da forma mais rápida e saborosa possível, é preciso contar com um processo bem-organizado. CI (Integração Contínua) é como um chef que, a cada pequena mudança no prato, reúne a equipe para experimentar juntos. Assim, erros são identificados rapidamente e a receita é ajustada antes de ser servida para todos os clientes. No desenvolvimento de software, a CI significa integrar as mudanças de código frequentemente, automatizando a construção e os testes, para garantir que tudo funcione como esperado. CD (Entrega Contínua) é como ter um sistema em que, a cada prato aprovado pelo chef e pela equipe, ocorre um envio automático do prato para a mesa dos clientes. No desenvolvimento, a CD automatiza o processo de entrega do software para os usuários, seja em um ambiente de testes ou diretamente para a produção.
- **Open-source (código aberto)**: imagine um livro de receitas que qualquer pessoa pode ler, copiar e até modificar. O código aberto funciona da mesma forma, mas com software. Trata-se de um software cujo código fonte está disponível para que qualquer pessoa possa visualizar, modificar e distribuir.
- **Contêiner:** é como uma caixa personalizada para transportar objetos. No mundo do software, um contêiner é um ambiente isolado, onde são empacotados um aplicativo e todas as suas dependências (bibliotecas, configurações etc.). Isso permite que o aplicativo seja executado de forma consistente em qualquer lugar, desde o seu computador até a nuvem.
- **Base de dados de séries temporais:** especializada em armazenar e recuperar dados que são coletados em intervalos de tempo específicos. Imagine um gráfico de temperatura ao longo de um dia: cada ponto do gráfico representa um dado em um momento específico.
- **Dashboards:** é como um painel de controle de um carro, mas para software. Apresenta de forma visual e organizada as informações mais

importantes sobre um sistema, como métricas de desempenho, gráficos e indicadores chave.

- **Histogramas e gauges:** histogramas são gráficos de barras que mostram a distribuição de um conjunto de dados. Por exemplo, um histograma pode mostrar quantas vezes cada valor de temperatura ocorreu em um determinado período. Gauges são medidores que mostram um valor numérico em relação a um limite máximo ou mínimo. Por exemplo, um gauge pode mostrar o uso da CPU de um servidor.
- **Build:** um build é o processo de transformar o código fonte de um software em um produto executável, como um aplicativo ou um site. Isso envolve compilar o código, empacotar as dependências e gerar um arquivo que pode ser executado.
- **API (Interface de Programação de Aplicativos)**: conjunto de regras e especificações que permitem que diferentes softwares se comuniquem entre si, como um menu em um restaurante. O cliente escolhe o que quer (uma função da API), e o restaurante (o software) prepara e entrega o prato escolhido.

2.2 Ferramentas para coletar e medir dados

No universo das ferramentas para desenvolvimento de software, duas categorias se destacam:

- **Ferramentas de gerenciamento de projetos e desenvolvimento:** essas ferramentas são utilizadas para planejar, organizar e acompanhar o progresso de projetos de software.
- **Ferramentas de análise e métricas:** essas ferramentas são utilizadas para coletar dados sobre o software, analisar seu desempenho e qualidade, e identificar áreas de melhoria.

Ao combinar essas ferramentas, as equipes de desenvolvimento podem potencializar a sua eficiência, elevando a qualidade dos produtos finais e reduzindo custos. A automação de tarefas repetitivas e a otimização dos processos agilizam o desenvolvimento, enquanto a identificação e a correção precoce de defeitos garantem a conformidade com os requisitos e padrões de qualidade. Além disso, a coleta e a análise de dados permitem tomar decisões

mais assertivas, baseadas em evidências, otimizando o uso de recursos e evitando retrabalhos.

Agora, entendidos os principais conceitos, vamos analisar algumas das ferramentas de coleta de métricas de software mais utilizadas atualmente, com suas características:

- **SonarQube:** é uma plataforma de código aberto que oferece análise estática de código para diversas linguagens de programação. O SonarQube identifica bugs, vulnerabilidades, "*code smells*" (sugerem a presença de problemas mais profundos, como um design ruim, lógica complexa ou até mesmo *bugs*) e violações de regras de codificação, fornecendo métricas sobre complexidade do código, cobertura de testes e duplicação de código, permitindo que as equipes melhorem continuamente a qualidade do seu software. **Exemplo de aplicação:** identificar duplicação de código, complexidade excessiva de métodos e classes, e possíveis bugs em um projeto Java (Sonarqube, 2025).
- **Jenkins:** ferramenta de integração contínua (CI) e entrega contínua (CD) amplamente utilizada. Embora não seja focada exclusivamente em métricas, o Jenkins pode ser configurado para coletar dados sobre *builds*, testes, cobertura de código e outros indicadores de desempenho. Ao integrar o Jenkins com outras ferramentas, é possível construir pipelines de CI/CD robustos e automatizados. **Exemplo de aplicação**: automatizar a construção, os testes e a implantação de um aplicativo web, garantindo que novas funcionalidades sejam entregues com rapidez e segurança (Jenkins, 2025).
- **Prometheus:** ferramenta de monitoramento e alerta open-source, especialmente projetada para sistemas em grande escala e ambientes de contêiner. Coleta métricas em tempo real e as armazena em uma base de dados de séries temporais, oferecendo visualização de dados através de gráficos e painéis personalizados, além de recursos de alerta para notificar sobre problemas críticos. **Exemplo de aplicação:** monitorar o uso de CPU e memória em um cluster de servidores, detectar gargalos de desempenho e gerar alertas em caso de falhas (Prometheus, 2025).
- **Grafana:** painel de visualização de dados open-source, que se integra com diversas fontes de dados, incluindo o Prometheus. O Grafana permite criar *dashboards* personalizados para monitorar métricas de

infraestrutura, aplicações e negócios. Oferece uma ampla variedade de visualizações, como gráficos de linha, histogramas, mapas e gauges. **Exemplo de aplicação:** visualizar métricas de um sistema distribuído, como o tempo de resposta de uma API, o número de requisições por segundo e a taxa de erro (Grafana, 2025).

- **JaCoCo:** ferramenta de cobertura de código para a plataforma Java. Gera relatórios detalhados sobre quais partes do código foram executadas durante os testes. O JaCoCo pode ser integrado a ferramentas de build, como o Maven e o Gradle, e seus resultados podem ser visualizados em formato HTML ou XML. **Exemplo de aplicação:** verificar se todos os casos de teste de um módulo estão cobrindo o código adequadamente (Jacoco).
- **Zabbix:** se destaca como uma ferramenta de **monitoramento** robusta e versátil, complementando o conjunto de ferramentas que vimos anteriormente. Enquanto o SonarQube foca na qualidade do código, o Jenkins na automação de processos, o Prometheus e o Grafana na coleta e visualização de métricas, o Zabbix tem como objetivo principal monitorar a integridade e o desempenho de toda a infraestrutura de TI.

A Figura 2 apresenta um comparativo entre as ferramentas de análise e métricas e sua utilização.

Figura 2 – Comparação entre ferramentas utilizadas para métricas de software

FERRAMENTA	VANTAGENS	DESVANTAGENS
SonarQube	Análise estática completa, diversas linguagens, comunidade ativa	Curva de aprendizado para configurações avançadas
Jenkins	Flexível, amplamente utilizado, grande quantidade de plugins	Configuração complexa para pipelines grandes
Prometheus	Escalabilidade, armazenamento de séries temporais, integração com Grafana	Requer conhecimento em sistemas distribuídos
Grafana	Visualizações personalizáveis, grande variedade de fontes de dados	Complexidade para criar dashboards complexos

JaCoCo	Cobertura de código detalhada, fácil integração com ferramentas de build	Focado em Java
Zabbix	Poderoso e versátil para monitoramento de infraestrutura	Curva de aprendizado e a complexidade em grandes ambientes

Fonte: Rosemari Pavan Rattmann, 2024.

Em um cenário de desenvolvimento de software cada vez mais complexo e exigente, a coleta e análise de métricas desempenham um papel fundamental para garantir a qualidade, o desempenho e a escalabilidade dos sistemas. Uma gama diversificada de ferramentas, cada uma com suas especificidades, colabora para fornecer uma visão holística do ciclo de vida do software, desde a concepção até a produção.

A utilização integrada das ferramentas de coleta de dados para métricas permite identificar gargalos de desempenho, otimizar recursos, garantir a qualidade do código e tomar decisões mais assertivas sobre o desenvolvimento e a manutenção do software. Ao integrar essas ferramentas, as equipes de desenvolvimento podem obter insights valiosos sobre o comportamento do sistema, prever problemas e tomar medidas proativas para garantir a satisfação dos usuários.

Novas tecnologias e metodologias surgem a todo momento, demandando uma atualização contínua das práticas e ferramentas utilizadas. A escolha das ferramentas mais adequadas dependerá das características específicas de cada projeto, do tamanho da equipe e das tecnologias envolvidas.

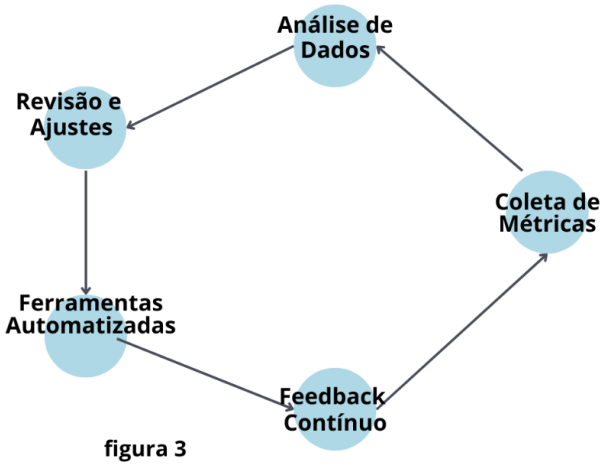
TEMA 3 – MELHORES PRÁTICAS NO USO DE MÉTRICAS

Para melhorar a qualidade e a utilidade das estimativas no desenvolvimento, compra ou manutenção de software, é essencial adotar práticas que garantam a sua precisão e relevância. Essas práticas ajudam a reduzir os riscos de erro na projeção de esforços, custos e prazos, fornecendo bases sólidas para a tomada de decisão.

A Figura 4 exemplifica o ciclo de melhores práticas para métricas de software, ilustrando como a coleta de métricas, a análise de dados, revisões e ajustes, o uso de ferramentas automatizadas e o feedback contínuo formam um

ciclo iterativo. Esse ciclo ajuda a garantir a melhoria contínua da qualidade do software, com base em métricas confiáveis e monitoramento constante.

Figura 3 – Ciclo interativo para métricas de software



A seguir, listamos algumas recomendações de melhores práticas:

- **Basear-se em métricas confiáveis:** o uso de métricas históricas e reais é fundamental para embasar estimativas sólidas, pois os dados provenientes de projetos anteriores ajudam a entender padrões, como o tempo necessário para implementar funcionalidades similares ou os custos médios associados a certas atividades (Pressman, 2010). Por exemplo, utilizar a métrica de produtividade por linha de código ou por ponto de função pode oferecer uma base quantitativa para projetar esforços futuros.
- **Adotar técnicas padronizadas de estimativa:** técnicas de estimativa amplamente reconhecidas, como Cocomo II (Constructive Cost Model) ou análise de pontos de função (FPA), ajudam a estruturar o processo de previsão. A padronização proporciona maior consistência e reduz a subjetividade nas estimativas (Sommerville, 2011):
    - **Cocomo II**: modelos matemáticos para calcular o esforço necessário com base em parâmetros como tamanho do software e complexidade do projeto.
    - **Pontos de função**: mede a complexidade funcional de um sistema, associando valores numéricos a funcionalidades e permitindo uma análise mais objetiva.

- **Incorporar revisões frequentes:** as estimativas devem ser tratadas como projeções dinâmicas, sendo revisadas regularmente à medida que novas informações e métricas se tornam disponíveis. Revisões contínuas ajudam a capturar mudanças no escopo, identificar riscos emergentes e ajustar cronogramas e orçamentos conforme necessário (Pressman, 2010). Essa prática é particularmente importante em projetos ágeis, em que o ciclo iterativo exige reavaliações constantes com base nas entregas realizadas e feedback dos stakeholders.
- **Realizar análise de cenários:** simular diferentes condições e variáveis, como aumento de equipe, mudanças nos requisitos ou alterações tecnológicas, é uma prática valiosa para entender os impactos potenciais no projeto. A análise de cenários permite antecipar problemas e preparar respostas para mitigar riscos (Sommerville, 2011). Por exemplo:
    - simular como uma mudança significativa nos requisitos pode afetar o cronograma ou custo;
    - avaliar os impactos de atrasos em etapas críticas do projeto.

A documentação e a transparência no uso de métricas de software são fundamentais para garantir análises confiáveis e rastreáveis. Métricas como linhas de código, cobertura de testes e complexidade do código oferecem uma base consistente para avaliar e comparar a qualidade do software ao longo do tempo. Essa prática não apenas alinha as equipes em torno de dados claros, mas também sustenta melhorias contínuas fundamentadas em informações precisas e acessíveis.

A colaboração interdisciplinar e o uso de ferramentas automatizadas, como SonarQube, amplificam os benefícios das métricas. A integração de perspectivas de desenvolvimento, qualidade e gestão permite identificar gargalos e priorizar melhorias com maior eficácia. Ferramentas automatizadas ajudam a monitorar métricas em tempo real, enquanto indicadores-chave, como redução de defeitos, orientam ajustes contínuos, promovendo entregas mais alinhadas aos objetivos do projeto e de maior valor.

Essas práticas não só aumentam a confiabilidade das decisões, mas também possibilitam a identificação de áreas de melhoria e a otimização de recursos ao longo do ciclo de vida do projeto, alinhando as entregas aos objetivos do negócio. A integração de métricas eficazes em cada etapa do

processo garante maior controle sobre a qualidade do software e promove um desenvolvimento mais ágil e sustentável.

TEMA 4 – FERRAMENTAS PARA ESTIMATIVAS DE SOFTWARE

A medição no desenvolvimento de software vai muito além de contar linhas de código ou avaliar a complexidade de funções. Acompanhar o progresso dos projetos, a produtividade das equipes e a qualidade dos processos é fundamental para garantir a entrega de software de alta qualidade, dentro do prazo. Existem diversas opções de ferramentas que podem auxiliar no gerenciamento e na medição do desenvolvimento de software e na aplicação das diversas técnicas de estimativas.

Em ambientes ágeis, a estimativa é um processo iterativo e colaborativo. O foco está no valor entregue ao cliente, e não apenas na quantidade de trabalho. As estimativas, sendo aproximadas, devem ser revisadas regularmente, e ajustadas conforme o projeto evolui e novas informações surgem. A participação de toda a equipe na estimativa é crucial para garantir que sejam realistas e reflitam o conhecimento coletivo da equipe sobre o projeto. Analisaremos algumas técnicas utilizadas para estimativas com metodologia ágil, com ferramentas que viabilizam e facilitam suas aplicações:

- **Planning Poker:** é uma técnica colaborativa que oferece diversos benefícios para a estimativa de tarefas em projetos ágeis. Ao promover a discussão e a troca de conhecimentos entre os membros da equipe, o Planning Poker facilita a construção de um consenso sobre o esforço necessário para cada tarefa. Ao realizar as estimativas de forma anônima, a técnica reduz o impacto de vieses individuais, garantindo estimativas mais precisas e objetivas. Além disso, o Planning Poker permite realizar as estimativas de forma rápida e eficiente, tornando o processo mais ágil e transparente para toda a equipe (Figura 4).
- **T-Shirt Sizing:** é uma técnica ágil para estimar o esforço necessário para completar uma tarefa**.** Utilizando tamanhos de camisetas como referência (P, M, G, GG), a equipe atribui um tamanho à tarefa, representando rapidamente se ela é pequena, média, grande ou extragrande. Essa abordagem simplificada e colaborativa permite uma estimativa inicial

rápida, ideal para quando a precisão não é a prioridade principal, mas sim a obtenção de uma visão geral do esforço envolvido em cada tarefa.

- **Story Points:** unidades de medida abstratas utilizadas em metodologias ágeis para quantificar a complexidade relativa de uma tarefa. Ao invés de medir o tempo diretamente, os Story Points servem como uma forma de comparar o esforço necessário para completar diferentes tarefas dentro de um projeto. Essa abordagem permite que as equipes estimem o trabalho de forma mais ágil e flexível, considerando não apenas a quantidade de trabalho, mas também fatores como complexidade técnica, incertezas e dependências entre as tarefas.

Figura 4 – Técnica Planning Poker

Crédito: DiPersika/Shutterstock.

Essas técnicas ajudam as equipes a entenderem melhor a complexidade das tarefas e a planejarem o trabalho de forma mais eficiente. Para facilitar a aplicação dessas técnicas, diversas ferramentas de software foram desenvolvidas, oferecendo recursos específicos para cada uma delas.

Exploraremos as principais ferramentas disponíveis no mercado, que dão suporte a essas técnicas de estimativa, abordando suas funcionalidades e vantagens e como escolher a ferramenta ideal. A seguir, identificaremos

algumas dessas ferramentas que dão suporte à utilização de técnicas de estimativas de software:

- **JIRA**: essencial para o gerenciamento de tarefas e bugs e o acompanhamento do progresso do projeto. Permite criar fluxos de trabalho personalizados, acompanhar o tempo gasto em cada tarefa e integrar-se com outras ferramentas. **Exemplo de aplicação:** gerenciamento de tarefas e *bugs*.
- **FPA (Function Point Analysis)**: métrica utilizada para estimar o tamanho de um software, baseada em sua funcionalidade. Permite comparar o tamanho de diferentes projetos e estimar o esforço necessário para o desenvolvimento. **Exemplo de aplicação:** medição completa de um software para aquisição ou avaliação do desenvolvimento e da equipe.
- **Azure DevOps**: plataforma completa para o desenvolvimento de software, que inclui ferramentas para planejamento, desenvolvimento, teste e implantação. Oferece métricas detalhadas sobre o desempenho das equipes e dos projetos. **Exemplo de aplicação:** gerenciamento completo do ciclo de vida do desenvolvimento.
- **Trello:** conhecido por sua interface simples e intuitiva, o Trello permite criar quadros Kanban personalizados para visualizar o fluxo de trabalho e estimar as tarefas utilizando as técnicas de T-Shirt Sizing ou Story Points. **Exemplo de aplicação**: criar quadros com colunas, como: “Ideias”, “Planejamento, “Em Produção” e “Finalizado”, para a equipe organizar as tarefas e responsabilidades.
- **GitHub**: além de ser um repositório de código, o GitHub oferece ferramentas para acompanhar o progresso do desenvolvimento, como gráficos de commits, issues e pull requests. versionamento de código. **Exemplo de aplicação:** controlar as versões geradas por diversos desenvolvedores.
- **Miro:** ferramenta de quadro branco digital colaborativo que pode ser utilizada para realizar sessões de Planning Poker online. Os participantes podem visualizar e votar em cartas virtuais, facilitando a colaboração à distância. **Exemplo de aplicação:** durante a fase de discussão do Planning Poker, a equipe utiliza o Miro para visualizar as cartas virtuais de estimativas já votadas.

- **Aha! Roadmaps:** focada em gestão de produtos, a Aha! Roadmaps oferece recursos para priorizar ideias, criar roadmaps e estimar o esforço necessário para implementá-las. A ferramenta suporta diferentes técnicas de estimativa, incluindo Planning Poker. **Exemplo de aplicação:** durante a etapa de priorização de ideias para um novo lançamento de produto, a equipe utiliza a ferramenta para organizar e ranquear ideias com base em critérios como impacto no cliente, alinhamento estratégico e esforço estimado.
- **Monday.com:** plataforma flexível para gerenciamento de projetos. Pode ser configurada para atender às necessidades específicas de cada equipe. Oferece recursos para planejamento ágil, incluindo estimativas e acompanhamento do progresso. **Exemplo de aplicação**: durante o planejamento de um projeto ágil, a equipe configura o Monday.com para criar um backlog de tarefas com prioridades claras. Cada tarefa é associada a uma coluna de estimativas, em que técnicas como Story Points são aplicadas para definir o esforço necessário. À medida que o projeto avança, os membros atualizam o status das tarefas e acompanham o progresso em painéis visuais personalizados. Isso permite identificar gargalos rapidamente, ajustar as prioridades de acordo com o feedback do cliente e garantir a entrega dentro dos prazos estabelecidos.
- **Asana**: outra ferramenta de gerenciamento de projetos ágil, que oferece recursos como criação de tarefas, acompanhamento de prazos e colaboração entre equipes. **Exemplo de aplicação:** gerenciamento de projetos ágeis.

Embora sejam amplamente utilizadas nesse contexto, a sua flexibilidade permite que sejam adaptadas a diferentes abordagens de gerenciamento de projetos, incluindo metodologias tradicionais

A Figura 5 apresenta uma comparação entre essas principais ferramentas de estimativas de software e suas aplicações.

Figura 5 – Comparativo entre as ferramentas de estimativas

Ferramenta	Principais funcionalidades	Exemplo de aplicação	Recursos adicionais
Trello	Gestão visual de tarefas com listas e cartões	Planejamento de sprints e estimativas de tarefas ágeis	Quadros Kanban, simplicidade de uso, integração com outras ferramentas
Asana	Gestão de tarefas, cronogramas e relatórios	Planejamento de projetos e acompanhamento do progresso	Cronogramas interativos, relatórios detalhados, integração com outras ferramentas
Monday.com	Gestão de projetos, fluxos de trabalho e automações	Planejamento de entregas de projetos ágeis com acompanhamento de progresso	Customização flexível, dashboards e automação de tarefas
Aha! Roadmaps	Criação e gerenciamento de roadmaps de produto	Priorização de funcionalidades e estimativas de recursos necessários para a implementação	Planejamento de produto, priorização de ideias, criação de roadmaps
Miro	Quadro branco colaborativo para visualização e planejamento	Realização de sessões de *planning poker online* com equipes distribuídas	Ferramenta colaborativa, personalização para workshops e brainstorms
GitHub	Repositório de código, controle de versão e colaboração	Acompanhamento do progresso de desenvolvimento e análise de código	Controle de versão, integração com outras ferramentas de CI/CD

Fonte: Rosemari Pavan Rattmann, 2024.

A escolha da ferramenta ideal para gestão de projetos depende de fatores como tamanho da equipe, integrações necessárias, orçamento disponível e funcionalidades oferecidas. Para equipes pequenas, ferramentas simples, como Trello, podem ser suficientes, enquanto equipes maiores podem exigir soluções mais robustas, como Jira. É fundamental que a ferramenta esteja alinhada à cultura da equipe, oferecendo recursos como relatórios e acompanhamento de progresso. Além disso, treinamentos para os usuários e revisões periódicas garantem uma adaptação contínua, especialmente em projetos ágeis, nos quais mudanças no escopo e prioridades são frequentes. Ao utilizar uma ferramenta

de estimativa, as equipes de desenvolvimento podem obter uma visão mais precisa do tempo, do custo e dos recursos necessários para completar um projeto, aumentando as chances de sucesso.

4.1 Uso de estimativas de software

As estimativas complementam as métricas, fornecendo projeções para apoiar decisões estratégicas. No desenvolvimento de software, elas ajudam a prever o esforço, o custo e o tempo necessários para alcançar um objetivo. Já na compra ou manutenção, as estimativas ajudam a avaliar o retorno do investimento e os custos de ciclo de vida, garantindo que os recursos sejam direcionados de forma eficiente. Embora as métricas ofereçam dados objetivos e mensuráveis, baseados em históricos e desempenho atual, as estimativas são ferramentas de projeção que complementam esses dados, ao prever aspectos futuros do projeto, como esforço, tempo e custo. A integração entre métricas e estimativas permite uma abordagem mais robusta para a tomada de decisões estratégicas, facilitando a análise de riscos e o alinhamento com os objetivos organizacionais.

4.1.1 Papel das estimativas no desenvolvimento

No desenvolvimento de software, as estimativas ajudam a prever o esforço necessário para completar tarefas específicas, o tempo para entrega de funcionalidades e os custos associados ao projeto. Por exemplo, técnicas como Planning Poker ou análise de pontos de função utilizam dados históricos (métricas) para fornecer projeções mais realistas. As estimativas são ferramentas fundamentais para criar cronogramas precisos, planejar alocação de recursos e estabelecer prioridades, especialmente em projetos ágeis, em que as entregas iterativas demandam previsões contínuas (Pressman, 2010).

4.1.2 Papel das estimativas na compra e manutenção

No caso de compra ou manutenção de software, as estimativas são utilizadas para calcular o retorno sobre investimento (ROI), os custos de ciclo de vida do sistema (Total Cost of Ownership – TCO) e os custos de integração e treinamento. Esses fatores são essenciais para comparar opções, como adquirir um software pronto ou desenvolvê-lo internamente.

A Figura 6 apresenta uma tabela comparativa entre métricas e estimativas de software.

Figura 6 – Comparativo do uso de métricas e estimativas de software

Aspecto	Métricas de Software	Estimativas de Software
Objetivo Principal	Fornecer dados quantitativos sobre o desempenho e a qualidade do software, ajudando a monitorar o progresso e identificar problemas.	Prever o esforço, custo e tempo necessários para concluir um projeto, baseando-se em dados e experiências anteriores.
Exemplo de Aplicação	Medir a complexidade do código, acoplamento entre módulos, ou o número de falhas durante o desenvolvimento.	Estimar o tempo necessário para finalizar uma funcionalidade, calcular o custo de desenvolvimento ou prever o retorno sobre o investimento.
Utilização na Tomada de Decisão	Auxilia na gestão de riscos, como mudanças de requisitos e impacto em sistemas legados. Revela áreas críticas no desenvolvimento que precisam de mais atenção.	Auxilia em decisões estratégicas, como escolher entre desenvolvimento interno ou compra de soluções prontas, considerando custos e prazos.
Impacto em Escolhas Estratégicas	Influencia decisões sobre onde investir esforços para melhorar a qualidade do software ou reduzir riscos de falhas.	Influencia decisões sobre alocação de recursos, definição de prazos e escolha entre diferentes abordagens de desenvolvimento.
Exemplos de Métricas/Estimativas	Taxa de defeitos, acoplamento entre módulos, cobertura de testes, complexidade ciclomática, tempo de resposta.	Tempo de desenvolvimento, custo por ponto de função, recursos necessários, Custo total de propriedade (TCO), COCOMO II

Fonte: Rosemari Pavan Rattmann, 2024.

A combinação de métricas sobre desempenho de soluções anteriores e estimativas sobre futuros investimentos facilita decisões fundamentadas, reduzindo os riscos de gastos excessivos ou de má adequação às necessidades do negócio.

Grandes empresas têm enfrentado desafios na estimativa de software e compartilhado lições aprendidas para melhorar esse processo. Vejamos alguns exemplos e melhores práticas:

- **Dell**: a Dell Technologies é uma das maiores empresas de tecnologia do mundo, com foco em soluções de hardware, software e serviços. Reorganizou sua equipe de marketing e TI para adotar métodos ágeis, o que incluiu a padronização das estimativas para sprints mensais. Essa abordagem aumentou a visibilidade do progresso e permitiu ajustes rápidos, minimizando discrepâncias nas entregas. O aprendizado aqui foi que a integração de ferramentas de rastreamento e a transparência na comunicação entre equipes são essenciais para estimativas confiáveis.
- **Google**: no gerenciamento do Google Chrome, a equipe utilizou técnicas de análise baseada em dados históricos e integração contínua, para prever com mais precisão os tempos de desenvolvimento. Um ponto crítico foi a adoção de ciclos de liberação mais curtos, permitindo revisões e replanejamentos frequentes.
- **Panera Bread**: é uma cadeia de padarias e cafés com foco em alimentos de alta qualidade e uma experiência acolhedora para os clientes. Fundada nos Estados Unidos, a empresa oferece uma ampla gama de produtos, que incluem sopas, saladas, sanduíches, massas, produtos de panificação frescos e bebidas. Para atender à demanda de crescimento rápido, a Panera implementou o framework Disciplined Agile Delivery (DAD) para melhorar o planejamento e a entrega de software, como o sistema de pedidos móveis. A empresa aprendeu que a adoção de metodologias iterativas melhora a precisão das estimativas, ao permitir ajustes rápidos baseados em feedbacks contínuos dos usuários finais.

Tanto as métricas quanto as estimativas são instrumentos importantes para a tomada de decisões eficazes no desenvolvimento de software. As métricas fornecem dados objetivos sobre o desempenho, permitindo acompanhamento contínuo e ajustes necessários durante o ciclo de vida do projeto. Já as estimativas projetam esforço, custo e tempo necessários, facilitando a alocação de recursos e o planejamento de prazos.

Embora ambas sejam fundamentais, elas servem a propósitos distintos. As métricas têm um caráter mais reativo, pois são usadas para medir o que já aconteceu e auxiliar em melhorias contínuas. Por outro lado, as estimativas têm um caráter mais preditivo, sendo utilizadas para planejar o futuro e tomar decisões antes de um projeto ser iniciado (ou em seus estágios iniciais).

Ao combinar essas duas abordagens de forma eficiente, com base em boas práticas e métricas confiáveis, os gerentes de projeto conseguem melhorar a precisão das decisões, reduzir os riscos e alinhar os resultados com os objetivos estratégicos do negócio. O uso adequado de métricas e estimativas é, portanto, um fator determinante para o sucesso de projetos de software.

FINALIZANDO

Ao longo desta abordagem, exploramos conceitos fundamentais e ferramentas práticas que compõem a base da gestão de projetos de software, com ênfase no uso de métricas e estimativas.

Inicialmente, examinamos as métricas e estimativas de software, reconhecendo sua importância para a redução de incertezas, a definição de prioridades e o acompanhamento do progresso. Vimos como essas ferramentas oferecem suporte às decisões estratégicas, utilizando dados objetivos e projeções estruturadas para minimizar riscos e otimizar recursos.

Na sequência, aprofundamos o nosso entendimento sobre as ferramentas para coleta e análise de métricas, identificando tecnologias que auxiliam em monitoramento da qualidade do código, desempenho dos sistemas e identificação de pontos de melhoria.

Avançando no conteúdo, discutimos as melhores práticas no uso de métricas, destacando a relevância de padronização, revisão contínua e análise qualitativa. Essas práticas foram analisadas como elementos fundamentais para garantir a confiabilidade dos dados e orientar decisões mais precisas em projetos complexos.

Em seguida, exploramos as ferramentas para estimativas de software, com foco na aplicação em metodologias ágeis. Investigamos como técnicas inovadoras, como Planning Poker e Story Points, promovem um planejamento colaborativo, dinâmico e alinhado às demandas dos projetos e das equipes.

Por fim, refletimos sobre o papel essencial das estimativas no desenvolvimento, compra e manutenção de software, e estudamos como a

integração entre métricas e estimativas permite prever custos, esforços e cronogramas de forma mais assertiva, alinhando estratégias tecnológicas aos objetivos organizacionais e de mercado.

Com este estudo, encerramos uma etapa significativa do aprendizado sobre métricas e estimativas no desenvolvimento de software. A aplicação prática desses conceitos, associada ao uso de ferramentas e melhores práticas, contribuirá diretamente para o sucesso em projetos futuros. Assim, concluímos que a gestão fundamentada em dados é um pilar essencial para a evolução e a excelência no campo tecnológico.

REFERÊNCIAS

ALBRECH, A. J.; GAFFNEY, J. E. Software Function, Source Lines of Code, and Development Effort Prediction: A Software Science Validation. **IEEE Transactions on Software Engineering**, 1983.

GRAFANA. Disponível em: <https://grafana.com/docs/>. Acesso em: 23 jan. 2025.

JACOCO. Disponível em: <https://www.jacoco.org/jacoco/trunk/doc/>. Acesso em: 23 jan. 2025.

JENKIS. Disponível em:<https://www.jenkins.io/doc/>. Acesso em: 23 jan. 2025.

KITCHENHAM, B.; PFLEEGER, S. L.; FENTON, N. Towards a Framework for Software Measurement Validation. **IEEE Transactions on Software Engineering**, 1996.

McCABE, T. J. A Complexity Measure. **IEEE Transactions on Software Engineering**, 1976.

PRESSMAN, R. S. **Engenharia de software***:* uma abordagem profissional. São Paulo: McGraw-Hill, 2010.

PROMETHEUS. Disponível em: <https://prometheus.io/docs/>. Acesso em: 23 jan. 2025.

SCHWABER, K.; BEEDLE, M. **Agile Software Development with Scrum**. São Paulo: Prentice Hall, 2002.

SOMMERVILLE, I. **Engenharia de Software**. São Paulo: Pearson, 2011.

SONARQUBE. Disponível em:<https://docs.sonarqube.org/latest/>. Acesso em: 23 jan. 2025.